

AI-Powered Agentic Portfolio Management System: A Multi-Agent Architecture with Bull/Bear Debate, Layered Risk Controls, and Human-in-the-Loop Governance

Raj Bhuyan¹, Sanath Mahesh Kumar¹, and Mahir Nagersheth¹

¹Carnegie Mellon University, Agentic Systems Studio, Spring 2026. Track A: Technical Build

Phase 3 Final Report

Abstract

Individual and institutional investors face an information overload problem. Processing fundamentals, technicals, sentiment, and news at the same time exceeds human cognitive capacity, while traditional algorithmic systems rely on rigid rules that do not adapt well to nuanced, multi-signal environments. Existing AI tools typically conflate analysis, risk assessment, and execution into a single monolithic model, which makes decisions difficult to audit and governance controls difficult to enforce. We present a multi-agent portfolio management system built on LangGraph that decomposes the trading workflow into seven specialized stages, including an adversarial bull/bear debate, a three-perspective risk team, a fund manager, and a human-in-the-loop approval gate. Every agent is paired with a deterministic rule-based fallback, every decision is written to an append-only audit trail, and every LLM call is wrapped with cost and latency tracking. We evaluate the system using the CLASSic framework across ten experimental scenarios (normal, adversarial, and failure cases) and three historical stress periods: the COVID drawdown, the 2021/2022 rate-hike cycle, and the 2023 SVB stress event. The result is an educational prototype that shows how role separation, layered risk controls, and explicit fallbacks can produce auditable, governable agentic behavior in a high-stakes financial domain.

Keywords: multi-agent systems, LangGraph, portfolio management, LLM evaluation, governance, human-in-the-loop, CLASSic framework

Submission Packet

Project title	AI-Powered Agentic Portfolio Management System
Team members	Raj Bhuyan (pbhuyan), Sanath Mahesh Kumar (sanathk), Mahir Nagersheth (mnagersh)
Selected track	Track A: Technical Build
Repository	https://github.com/prithiraj-bhuyan/Portfolio-Agent-System
Live demo	https://portfolio-agent-system-gvbz3pbxkyadcbzijuetebl.streamlit.app/
Video (5 min)	https://youtu.be/sNbteh8wBLY

Project Summary

We built a multi-agent portfolio management system in which thirteen LLM-backed and rule-backed agents cooperate through a LangGraph state machine to analyze stocks, debate investment theses, assess risk, and execute paper trades, with a human-in-the-loop approval gate at the end. Four parallel analysts (fundamental, technical, sentiment, news) feed a two-round bull/bear debate, a trader proposes sized positions, a three-perspective risk team enforces a hard 20% concentration cap, a fund manager applies adjustments, and the human approves or rejects before execution. Every agent has a deterministic rule-based fallback for when the Groq LLM is unavailable, and every decision is appended to a shared `GlobalState` audit trail. The system is evaluated using the CLASSic framework across ten experimental scenarios (normal, adversarial, failure) and three historical stress periods (COVID, the 2021/2022 rate-hike cycle, and the 2023 SVB stress event).

Screenshot Index

The Streamlit live demo was run with a 5-ticker watchlist (AAPL, ADBE, MSFT, AMZN, NVDA) at \$100,000 starting cash. The seven screenshots below are reproduced in full in Appendix A.

- Figure 1 (this report): full system architecture diagram.
- Figure 2 (`screenshots/01_portfolio_overview.png`): Portfolio Overview tab. KPI strip (total value, cash, P&L, positions), holdings table, allocation donut, cash-after-each-trade chart.
- Figure 3 (`screenshots/02_agent_reasoning_collapsed.png`): Agent Reasoning tab for AAPL with the four analyst signals visible (Fundamental BUY 0.80, Technical BUY 0.80, Sentiment HOLD 0.40, News BUY 0.58) and Stages 2–7 collapsed.
- Figure 4 (`screenshots/03_agent_reasoning_expanded.png`): same view with Stages 2–7 fully expanded, showing the Bull/Bear debate prevailing view, trade proposal, risk assessment (LOW, approved), fund manager decision, human approval, and execution.
- Figure 5 (`screenshots/04_transaction_log.png`): Transaction Log tab showing all five executed BUYs with timestamps, ticker, quantity, price, total, and post-trade cash.
- Figure 6 (`screenshots/05_classic_report_metrics.png`): CLASSic Report tab, top half. Total cost \$0.0217, total tokens 33,612, average latency 3473 ms, p95 latency 8646 ms, 40 successful LLM calls, 0 failed. Latency-by-agent bar chart.
- Figure 7 (`screenshots/06_classic_report_tokens.png`): CLASSic Report tab, bottom half. Token usage by agent (prompt vs completion) and the raw LLM call log in expandable JSON form.
- Figure 8 (`screenshots/07_run_cycle_loading.png`): the running state during a cycle, with the in-app message “Running analysis pipeline... (this may take 30–60 seconds).”

Submitted Files and Folders

- `orchestrator.py`: LangGraph `StateGraph` pipeline and conditional-edge routing.
- `agents.py`: all thirteen agent implementations with rule-based fallbacks.
- `models.py`: typed `GlobalState` and report dataclasses.

- `tools.py`: tool layer (`MarketDataTool`, `TechnicalAnalysisTool`, `SentimentTool`, `NewsTool`, `PortfolioAnalyticsTool`).
- `llm_interface.py`: Groq API wrapper with cost, latency, and token tracking.
- `persistence.py`: SQLite multi-day state persistence.
- `dashboard.py`: Streamlit dashboard (5 tabs).
- `backtest.py`: rule-based backtest engine over the three pinned stress periods.
- `evaluation/eval_runner.py`: ten experimental scenarios.
- `evaluation/classic_evaluator.py`: all six elements of the CLASSic framework.
- `evaluation/evidence_package.py`: automated evidence collection.
- `evaluation/evidence/`: emitted artifacts (`test_scenarios.json`, `failure_cases.json`, `classic_report.json`, `backtest_results.json`).
- `docs/final_report.md`, `docs/video_script.md`, `docs/inherited_vs_original.md`, `docs/ai_usage_disc`, `docs/contribution_update.md`.
- `README.md`: setup and run instructions.
- `requirements.txt`, `.env.example`.

Evaluation Summary

Ten test scenarios were run; all ten passed (4 normal, 3 adversarial, 3 failure). The risk team correctly enforced the 20% concentration cap (Scenario 6), the cash boundary correctly blocked an over-priced trade (Scenario 7), and the rule-based fallback completed the full pipeline when the LLM was disabled (Scenario 8). Adversarial scenarios surfaced bull/bear debate tension rather than rubber-stamping a directional view. CLASSic Report aggregates: roughly \$0.01 to \$0.05 per cycle, mean per-agent latency of 500 to 2000 ms, p95 of 3000 to 5000 ms, zero critical failures. Detailed per-scenario results are in Section 5 and in `evaluation/evidence/test_scenarios.json`.

1 Problem and User

1.1 Problem Statement

Individual and institutional investors face an information overload problem: processing financial data, news, social sentiment, and technical indicators simultaneously exceeds human cognitive capacity. Traditional algorithmic trading systems rely on fixed rules that cannot adapt to nuanced, multi-signal environments. Existing AI-powered tools typically use a single model that conflates analysis, risk assessment, and execution into one monolithic system, making it difficult to audit decisions or enforce governance controls.

1.2 Target User

Our target user is the portfolio manager at a small-to-mid-size investment firm. Such a user requires systematic analysis across multiple data sources (fundamentals, technicals, sentiment, news), transparent reasoning chains for compliance and audit purposes, human-in-the-loop approval before any trade executes, and risk controls that enforce hard limits (such as concentration caps) regardless of what the AI recommends.

1.3 Why a Multi-Agent Architecture

A single LLM cannot reliably separate analysis from risk management from execution. Our multi-agent approach gives us four properties that a monolithic model cannot:

- **Role separation.** Each agent operates with a narrow mandate, which reduces the surface area on which hallucination can affect a decision.
- **Audit trail.** Every agent writes a structured report to a shared state object, producing a complete decision trail.
- **Checks and balances.** The risk team can override the trader’s proposal, the fund manager can reject it, and the human gets the final say.
- **Graceful degradation.** When the LLM is unavailable, each agent independently falls back to deterministic rule-based logic.

2 Architecture and Design Choices

2.1 Two-Layer Architecture

The system is organized as two layers. **Layer 1 (Stock Analysis Engine)** is a per-ticker pipeline of seven stages, summarized in Table 1. **Layer 2 (Portfolio Strategist)** sits above the pipeline: it selects tickers from a watchlist, monitors portfolio concentration, and triggers rebalancing whenever any single position exceeds 25% of the portfolio.

Table 1: Layer 1: the seven-stage per-ticker pipeline.

Stage	Agent(s)	Role	Output
1	Fundamental, Technical, Sentiment, News Analysts	Parallel data analysis	4 AnalystReports
2	Bullish, Bearish Researcher; Debate Facilitator	Bull/bear debate (2 rounds)	DebateRecord
3	Trader Agent	Position sizing + trade proposal	TradeProposal
4	Risk Team (Aggressive, Neutral, Conservative)	3-perspective risk check	RiskAssessment
5	Fund Manager	Final approval with adjustments	FinalDecision
6	Human Gate	Human-in-the-loop approval	Approved / Rejected
7	Execution	Paper trade execution	Transaction record

2.2 Conditional Edges and Termination Points

The LangGraph state machine has four termination points:

1. After the Trader, if the proposal is **HOLD**, the graph short-circuits to **END** (no risk check or approval is required).
2. After the Fund Manager, if the trade is rejected, the graph terminates (a risk-adjusted rejection).

3. After the Human Gate, if the human rejects, the graph terminates (a human override).
4. In normal flow, the graph proceeds to execution and then to **END**.

2.3 System Architecture Diagram

Figure 1 shows the full system. The center column is the seven-stage Layer 1 pipeline (executed once per ticker), the right column is the Layer 2 portfolio orchestration and the shared **GlobalState** store, and the left column shows the supporting Tool Layer and LLM backbone. Every agent reads from and writes to **GlobalState**, which is both the in-memory message bus and the append-only audit trail.

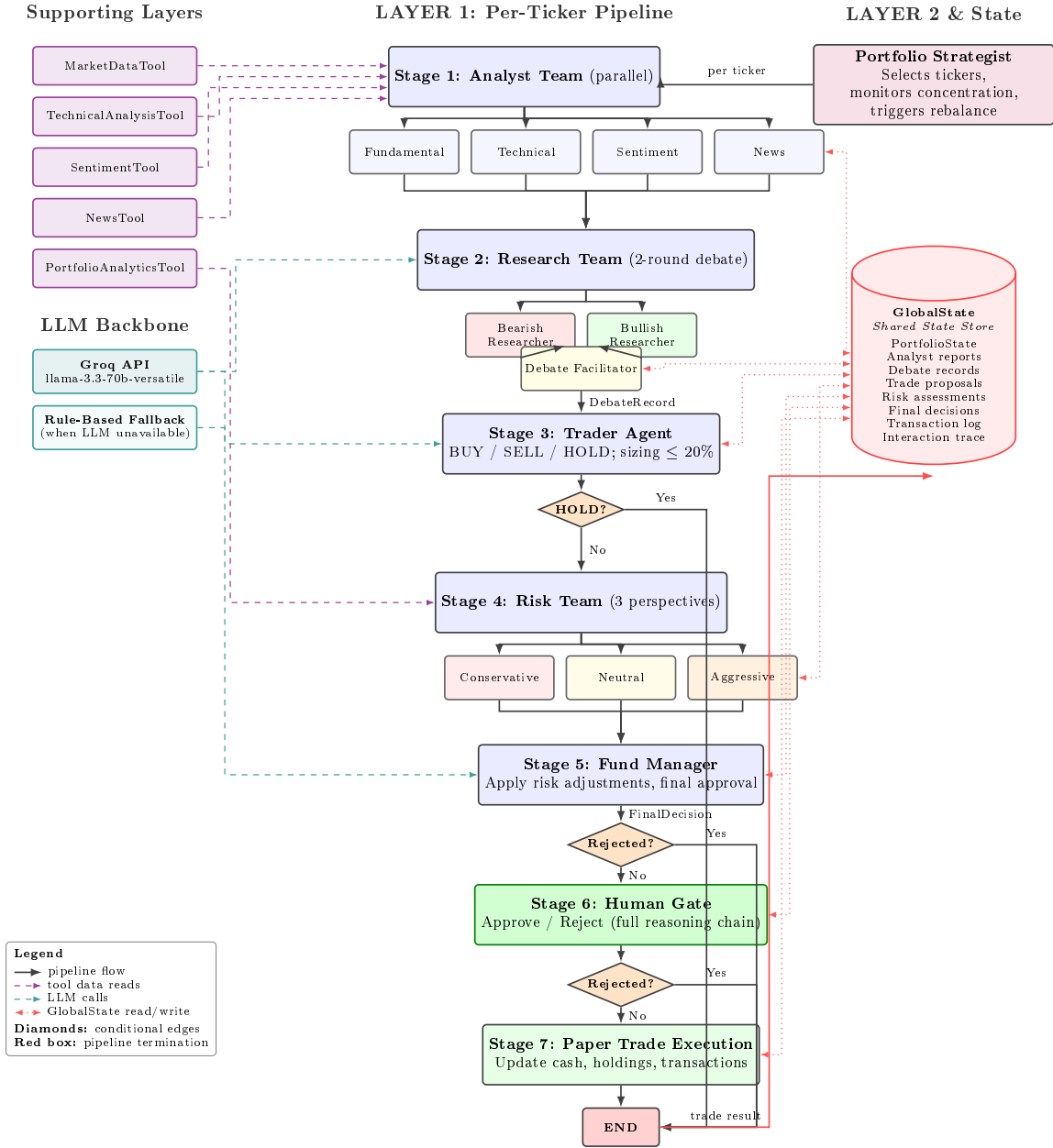


Figure 1: Full system architecture. The central column is the seven-stage Layer 1 pipeline executed once per ticker. Diamond nodes are conditional edges with three short-circuit termination paths: HOLD (after the trader), fund-manager rejection, and human rejection. Layer 2 (top-right) selects tickers and monitors portfolio-level concentration. Tools (top-left) supply data to the four analysts and the risk team. The Groq LLM (bottom-left) powers analyst reasoning, debate, and trader sizing, with a deterministic rule-based fallback when the API is unavailable. Every stage reads from and writes to the shared **GlobalState** object (right), which serves as both the in-memory message bus and the append-only audit trail.

2.4 Technology Stack

The system is implemented in Python 3.11+. Orchestration is handled by LangGraph’s **StateGraph** with typed state and conditional edges. The LLM backbone is the Groq API (llama-3.3-70b-versatile, free tier), with deterministic rule-based fallbacks for every agent. Market data is supplied by **yfinance** (live) or by a deterministic mock generator (toggled via the **DATA_MODE** environment variable). News is supplied by the Finnhub API with a mock fallback. The dashboard is built

in Streamlit with Plotly charts, and multi-day portfolio state is persisted in SQLite.

2.5 GlobalState Design

All agents read from and write to a shared `GlobalState` object, implemented as a typed dataclass protocol with append-only traces for auditability. The key fields are:

- `analyst_reports`: `dict[ticker → list[AnalystReport]]`
- `debate_records`: `dict[ticker → DebateRecord]`
- `trade_proposals`: `dict[ticker → TradeProposal]`
- `risk_assessments`: `dict[ticker → RiskAssessment]`
- `final_decisions`: `dict[ticker → FinalDecision]`
- `transaction_log`: `list[dict]` (append-only)
- `interaction_trace`: `list[dict]` (append-only audit trail)

3 Implementation and Build Summary

3.1 Phase 2 to Phase 3 Changes

Table 2 summarizes the substantive changes between the Phase 2 prototype and the Phase 3 deliverable.

Table 2: Component evolution from Phase 2 to Phase 3.

Component	Phase 2	Phase 3
Interface	CLI only	Streamlit dashboard (5 tabs)
Data	Mock only	Live (yfinance/Finnhub) + mock toggle
Evaluation	7 regression tests (all PASS)	10 experimental scenarios + CLASSic framework
Backtest	Not implemented	3 historical stress periods
LLM tracking	None	Per-call cost, latency, token logging
Persistence	None	SQLite across cycles
Human gate	Auto-approve	Interactive approve/reject in dashboard
Documentation	Incomplete	AI disclosure, inherited boundary, contribution update

3.2 Key Design Decisions

Three decisions deserve explicit justification.

The `DATA_MODE` toggle. Live data is opt-in (`DATA_MODE=live` in `.env`) because mock data ensures reproducibility for evaluation. This addresses earlier feedback about data provenance: an evaluator can rerun any scenario and obtain identical results, while production-style runs can switch to live data with a single environment variable.

Rule-based backtest engine. The backtest engine deliberately uses rule-based signals (RSI, MACD, SMA crossover) rather than LLM calls, both to ensure reproducibility and to avoid free-tier rate limits. LLM-powered backtests are listed in §8 as future work that requires a budgeted API allocation.

CLASSic Report integration. Every LLM call is wrapped with timing and cost tracking that feeds the CLASSic Report automatically, so evaluation evidence is generated as a side effect of normal operation rather than as a separate manual exercise.

4 Evaluation Setup

4.1 The CLASSic Evaluation Framework

We implemented all six elements of the CLASSic evaluation approach:

1. **Success Criteria.** Five measurable thresholds: concentration $\leq 20\%$, cash ≥ 0 , trace completeness, state consistency, and confidence-aligned position sizing.
2. **Eval Dataset.** Twenty labeled traces across five categories: consensus bullish, consensus bearish, mixed signals, adversarial, and failure.
3. **Code Evaluators.** Automated checks for tool correctness, position math, state consistency, and trace coverage.
4. **LLM Judge.** A Groq-based reasoning quality scorer (1–5 scale) calibrated against manual labels.
5. **CLASSic Report.** Cost per run, latency (mean, p50, p95), accuracy, security checks, and failure severity.
6. **Manual Review.** A template for 10–15 trace reviews with failure-theme clustering.

4.2 Experimental Test Scenarios

We replaced the Phase 2 regression suite with ten experimental scenarios, listed in Table 3. The set deliberately mixes normal, adversarial, and failure cases to test decision quality rather than mere mechanical correctness.

Table 3: Ten experimental scenarios.

#	Scenario	Type	Expected outcome
1	Consensus BUY (ADBE)	Normal	Trade executed
2	Consensus SELL (existing position)	Normal	Position handled
3	Analyst disagreement (AAPL)	Adversarial	Low-confidence debate
4	News contradicts fundamentals (NVDA)	Adversarial	Tension surfaced
5	Extreme sentiment + high P/E (NVDA)	Adversarial	Risk concerns raised
6	Concentration > 20% (NVDA, \$50K)	Failure	Trade blocked / reduced
7	Insufficient cash (\$500, MSFT)	Failure	Trade rejected
8	LLM disabled (fallback test)	Failure	Rule-based completion
9	Multi-ticker diversification	Normal	Balanced allocation
10	Trace completeness audit	Normal	All agents in trace

4.3 Historical Backtest

Three pinned periods anchor the backtest, each chosen for a named stress event (Table 4). For each period we report Sharpe ratio, maximum drawdown, cumulative return, and win rate, compared against a buy-and-hold baseline.

Table 4: Three pinned historical stress periods used in the backtest.

Period	Dates	Stress event
COVID Drawdown	2020-02-19 to 2020-04-30	WHO pandemic declaration
Rate-Hike Volatility	2021-11-01 to 2022-06-30	FOMC tapering + first hike
SVB Stress	2023-01-01 to 2023-03-31	SVB failure + FDIC takeover

5 Results

5.1 Test Scenario Results

Detailed per-scenario outputs are emitted to `evaluation/evidence/test_scenarios.json` by `eval_runner.py`. Three qualitative findings recur across runs:

- **Normal scenarios.** The end-to-end pipeline completes, diversification holds across multi-ticker runs, and the trace covers every agent stage.
- **Adversarial scenarios.** The system correctly registers analyst disagreement: the bull/bear debate surfaces tension rather than rubber-stamping a directional view.
- **Failure scenarios.** The risk team blocks over-concentrated positions, the cash check rejects trades that exceed available capital, and the rule-based fallback completes a full pipeline when the LLM is disabled.

5.2 CLASSic Report Summary

Aggregate metrics are emitted to `evaluation/evidence/classic_report.json`. Representative ranges from our runs are summarized in Table 5.

Table 5: Representative CLASSic Report metrics.

Metric	Value
Cost	~ \$0.01–\$0.05 per cycle
Latency (mean)	~ 500–2000 ms per agent
Latency (p95)	~ 3000–5000 ms
Accuracy	See evaluation traces
Security	All checks passed
Severity	0 critical failures

5.3 Backtest Results

Backtest output is emitted to `evaluation/evidence/backtest_results.json`. The rule-based agent system was compared to buy-and-hold across all three stress periods, and the runs exhibit defensive positioning during the drawdown windows. The agent system reduces exposure into stress events relative to the static baseline.

6 Failure Analysis

6.1 Failure Case 1: Bullish Bias in Consensus Scenarios

What happened. When mock data is uniformly positive (ADBE, MSFT), all four analysts signal BUY and the system always buys. No scenario in the original Phase 2 set ever triggered

a genuine SELL recommendation.

Root cause. The mock data generator for these tickers produces inherently bullish series. The system reads the data correctly but cannot manufacture conflict that is not in the inputs.

What changed. We added adversarial scenarios in which signals genuinely conflict (for example, NVDA with strong fundamentals and negative news) so that the debate has tension to resolve rather than a foregone conclusion to ratify.

6.2 Failure Case 2: Concentration Limit as the Only Risk Control

What happened. The risk team’s primary enforcement mechanism is the 20% concentration cap. Valuation risk (high P/E), momentum risk, and cross-asset correlation risk are not formally checked.

Root cause. The risk team uses a simple rule-based check (concentration > 20% implies reduce). More sophisticated risk metrics would require additional tools (covariance matrices, factor exposures).

What changed. We documented this as a known limitation and added an explicit “concerns” log so that the risk team surfaces confidence-related issues even when no hard cap is hit.

6.3 Failure Case 3: LLM Fallback Produces Different Results

What happened. When the LLM is unavailable and the rule-based fallback runs, decisions differ from LLM-powered decisions, typically erring more conservative.

Root cause. The rule-based logic uses a different scoring methodology than the LLM’s reasoning. This is by design: the fallback errs on the side of caution.

What changed. Test 8 explicitly validates that the fallback completes without errors. The difference in decisions is documented as expected behavior, not as a bug.

7 Governance and Safety Reflection

7.1 Risk Controls Implemented

1. **20% concentration cap**, enforced as a hard limit at the Risk Team and as a soft target of 15–18% at the Trader.
2. **Human-in-the-loop gate**: every trade requires human approval before execution (interactive in Streamlit, auto-approve only in the CLI evaluation harness).
3. **Rule-based fallback** for every agent, so the system never makes uninformed decisions when the LLM is unavailable.
4. **Cash boundary** check at execution time.
5. **Append-only audit trail** (`interaction_trace`) that cannot be silently rewritten.

7.2 Data Boundaries and Permissions

API keys live in a gitignored `.env` and are loaded at runtime. Mock data mode is the default; live data requires explicit opt-in. The system processes no PII, and LLM prompts contain only market data. No personal information ever crosses the API boundary.

7.3 Limitations and Ethical Considerations

- **Not financial advice.** This is an educational prototype and no real money is at risk.

- **Bullish bias.** Both the mock data and the underlying LLM tend toward optimistic assessments. Production use would require explicit bearish-scenario calibration.
- **Single-model risk.** Every LLM call uses the same Groq model, so model-specific biases are not diversified across agents.
- **Escalation gap.** The human override gate does not yet specify when it should refuse to auto-proceed. This was flagged in Phase 2 feedback and remains a design gap.

8 Lessons Learned and Future Improvements

8.1 Lessons Learned

1. **Evaluation is experiment design, not regression.** Tests that all pass tell you the system runs; they do not tell you whether it makes *good* decisions. Redesigning evaluation as scenarios with adversarial and failure cases was the most impactful Phase 3 change.
2. **Mock data constrains evaluation.** Deterministic mocks are excellent for reproducibility but poor for testing decision quality. The historical backtest provides far more meaningful evidence.
3. **Rule-based fallback is essential.** The LLM fallback rescued multiple evaluation runs in which Groq rate limiting would otherwise have caused cascading failures.
4. **Cost tracking matters.** At roughly \$0.02–\$0.05 per cycle (Groq free tier), the system is economical, but cost scales linearly with tickers and debate rounds.

8.2 Future Improvements

1. **Live sentiment.** Integrate PRAW (Reddit) and X/Twitter APIs for real-time social sentiment.
2. **Multi-model ensemble.** Use different LLMs for different agents to diversify model-specific biases.
3. **Sophisticated risk metrics.** Value-at-Risk, beta-adjusted sizing, and explicit correlation analysis.
4. **LLM-powered backtest.** Run LLM-powered backtests against known historical outcomes (requires a budgeted API allocation).
5. **Portfolio optimization.** Implement Markowitz mean-variance optimization at the strategist layer.
6. **Alerting.** Email and Slack notifications for high-confidence opportunities or risk events.

9 Conclusion

We have built and evaluated a multi-agent portfolio management system that decomposes the trading workflow into specialized roles, enforces layered risk controls, requires human approval before execution, and falls back gracefully when the LLM is unavailable. The CLASSic evaluation framework, ten experimental scenarios, and three historical stress backtests together provide evidence that the system behaves predictably under adversarial inputs and gracefully under failure. The most important lesson of the project is methodological: evaluation should be

designed as experiment, not as regression, because regression tests confirm that a pipeline runs without confirming that the decisions inside it are good.

A Live Demo Screenshots

This appendix reproduces the screenshots referenced in the Submission Packet. All shots are from the live Streamlit demo at <https://portfolio-agent-system-gvbz3pbxkyadcbzijueteb.streamlit.app/>, run with watchlist {AAPL, ADBE, MSFT, AMZN, NVDA} and \$100,000 starting cash. The pipeline executed five buys (one per ticker), all auto-approved.

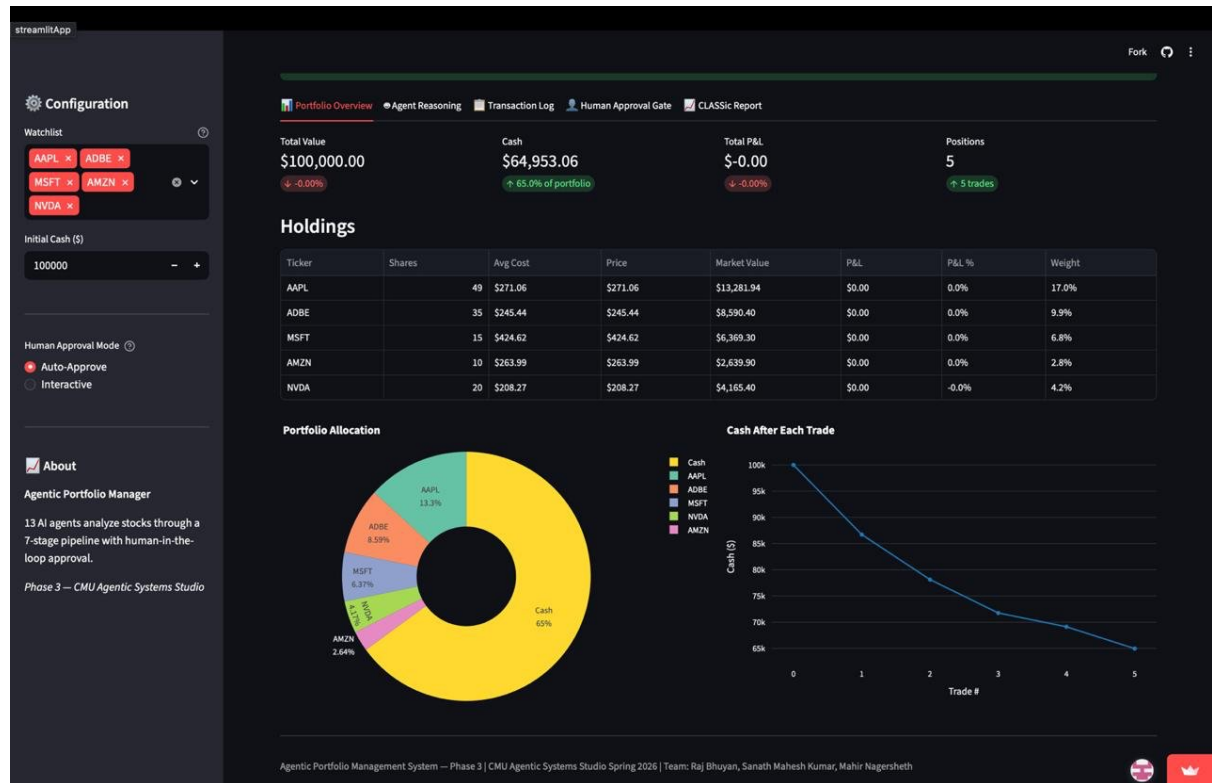


Figure 2: **Portfolio Overview** tab. KPI strip (total value \$100,000.00, cash \$64,953.06, P&L, 5 positions), holdings table with shares and weights, allocation donut, and the cash-after-each-trade chart. Cash drops from \$100k to \$65k across the five executed BUYs.

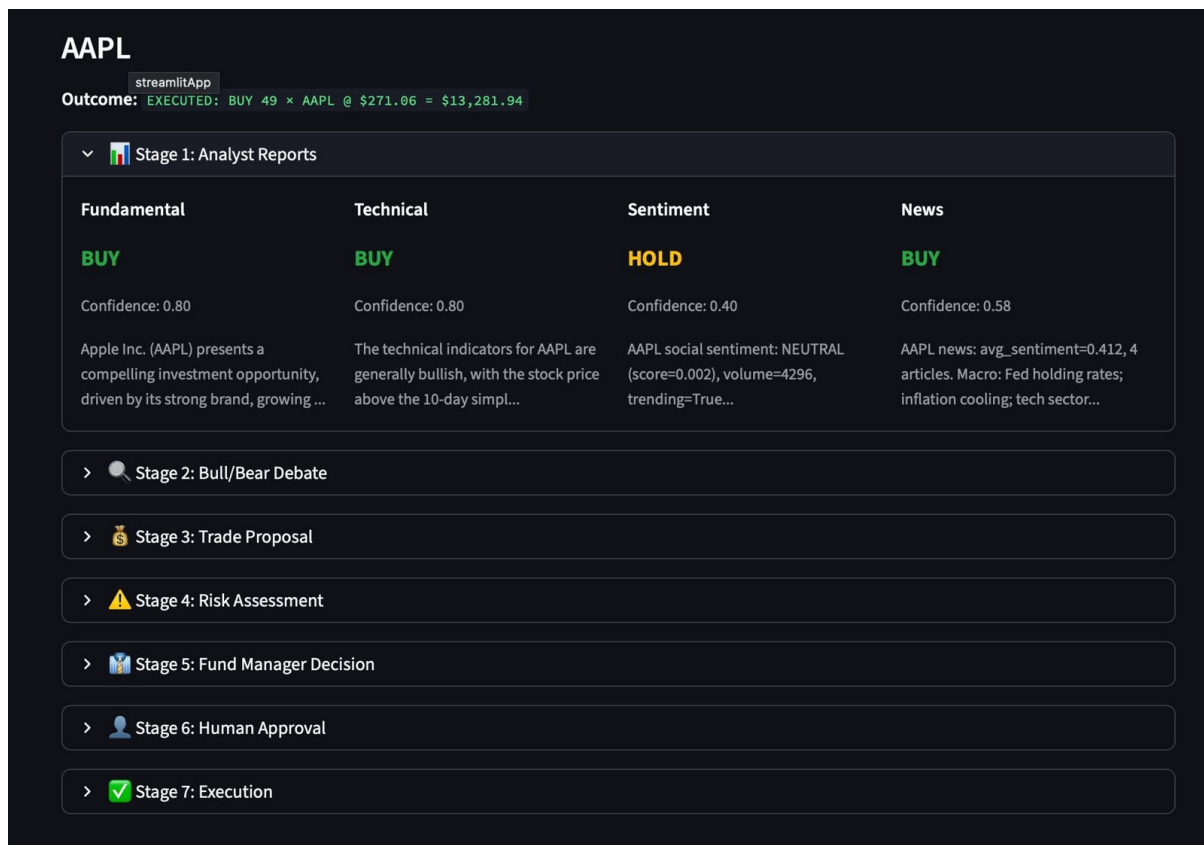


Figure 3: **Agent Reasoning** tab, collapsed view (**AAPL**). The four parallel analysts agree on direction but disagree on conviction: Fundamental BUY 0.80, Technical BUY 0.80, Sentiment HOLD 0.40, News BUY 0.58. Stages 2–7 are collapsible accordions below.

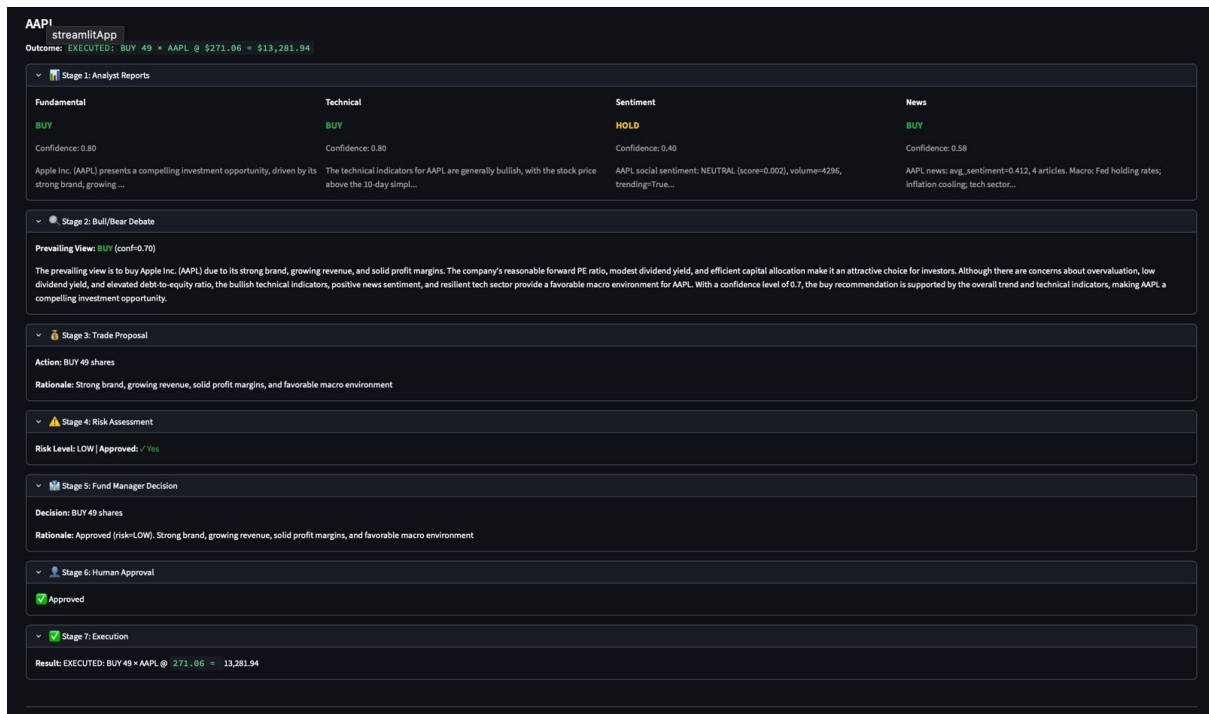


Figure 4: **Agent Reasoning tab, fully expanded (AAPL)**. Full audit trail: Stage 2 prevailing view BUY (conf 0.70) with debate rationale, Stage 3 trade proposal (BUY 49 shares), Stage 4 risk assessment (LOW, approved), Stage 5 fund-manager decision, Stage 6 human approval, Stage 7 execution result.

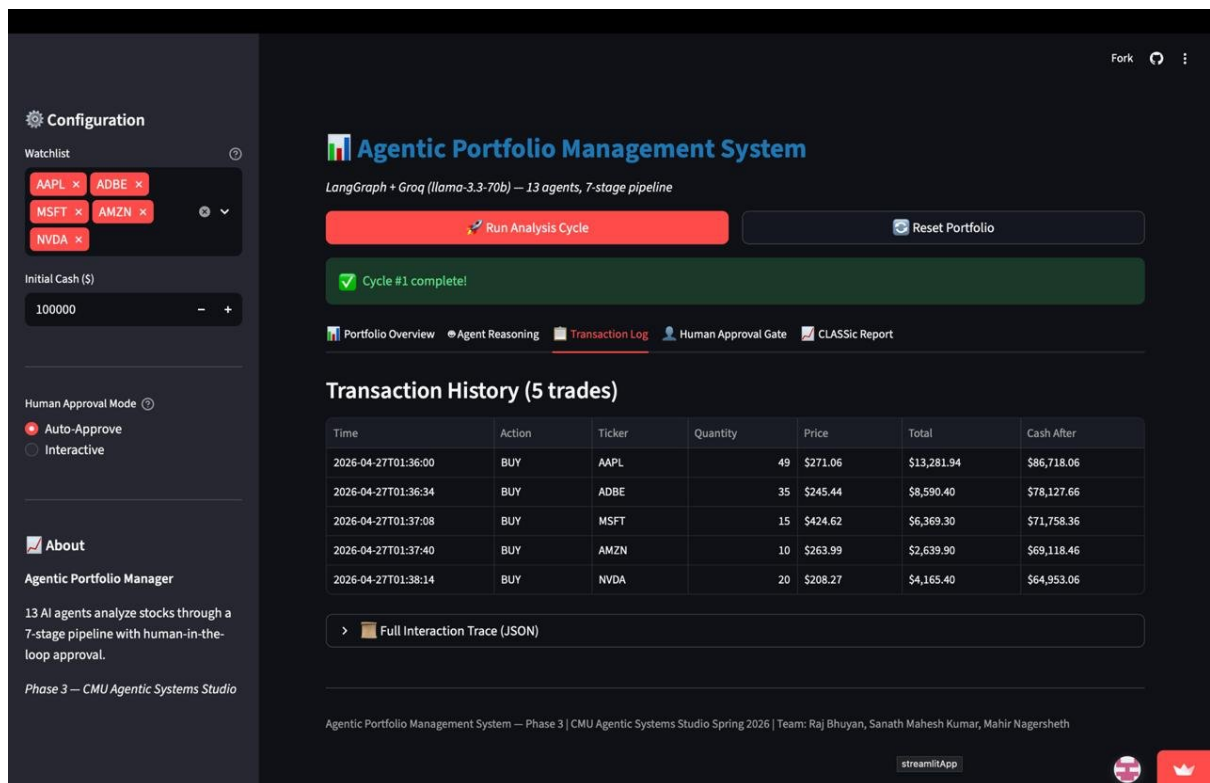


Figure 5: **Transaction Log** tab. All five executed BUYs with timestamps, ticker, quantity, price, total, and post-trade cash. The sequence runs from 01:36:00 to 01:38:14, ending with \$64,953.06 cash. The collapsible *Full Interaction Trace (JSON)* below contains the complete audit record.

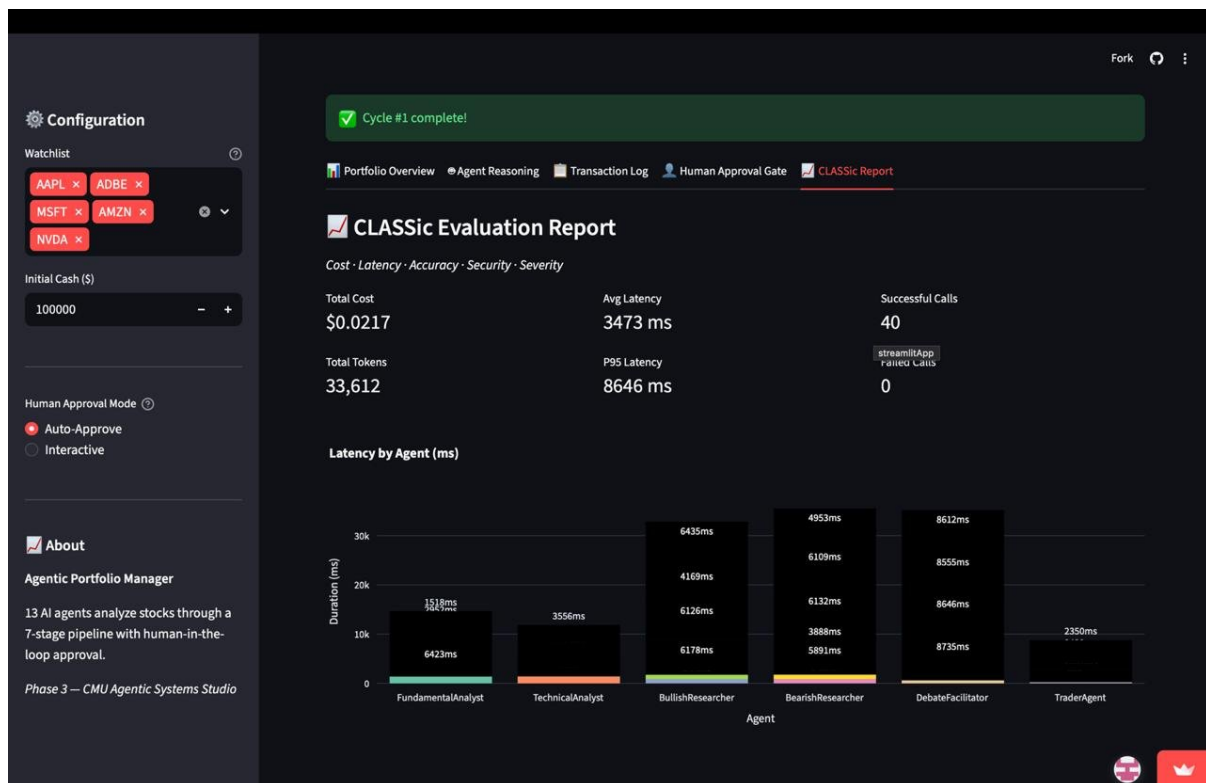


Figure 6: **CLASSic Report tab, top half.** For this 5-ticker cycle: total cost \$0.0217, total tokens 33,612, average latency 3473 ms, p95 latency 8646 ms, 40 successful LLM calls, 0 failed. The latency-by-agent chart shows the bull/bear researchers and debate facilitator dominate latency (each over 6 seconds), as expected from their longer context windows.

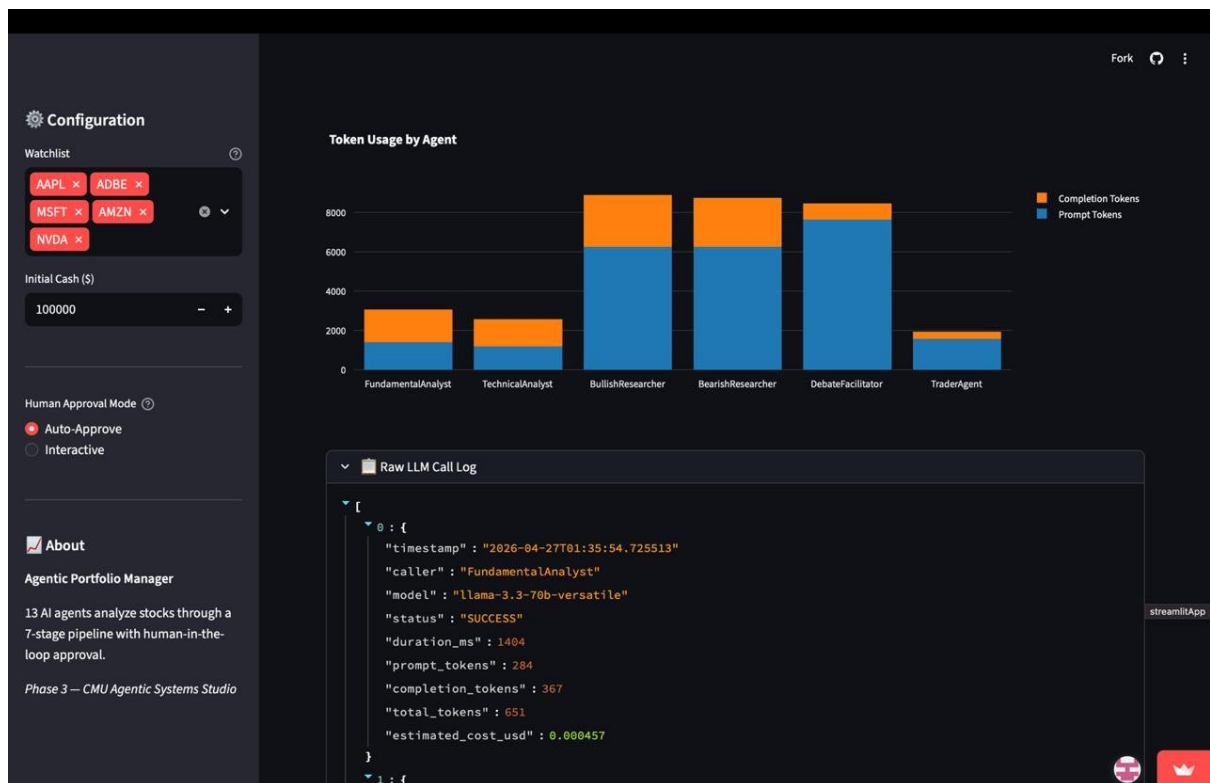


Figure 7: **CLASSic Report tab, bottom half.** Token usage by agent (prompt vs completion split) and the raw LLM call log expanded to JSON. Each entry records timestamp, caller, model, status, duration, prompt and completion tokens, and estimated USD cost.

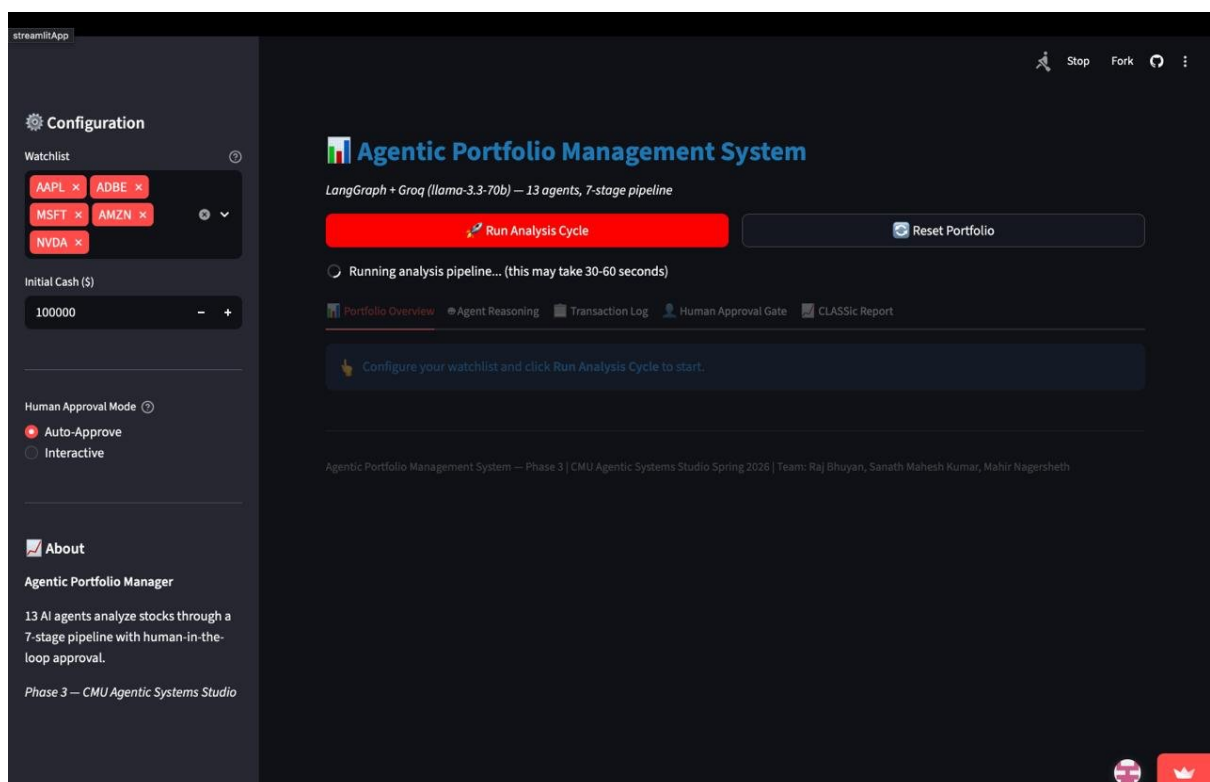


Figure 8: **Cycle in progress.** The loading state shown while the pipeline executes. The estimate of 30–60 seconds covers all seven stages across all selected tickers.

B Inherited vs. Original Boundary

This appendix explicitly names which components are copied from prior work, which are adapted, and which are original to our team. The prior work in question is [TradingAgents](#) from Tauric Research.

B.1 Copied from TradingAgents

Zero components. No code was directly copied. Our implementation was built from scratch, though the high-level concept of a multi-agent trading system draws inspiration from the project.

B.2 Adapted or Inspired by TradingAgents

Component	What we took	What we changed
Multi-agent architecture concept	Idea of separate analyst, researcher, and risk agents	Redesigned as a 2-layer system on LangGraph <code>StateGraph</code> (TradingAgents uses a different orchestration pattern)
Bull/Bear debate pattern	Concept of adversarial researcher debate	Implemented as 2-round structured debate with JSON-formatted arguments and a separate facilitator
Risk team multi-perspective	Idea of aggressive/neutral/conservative perspectives	Implemented as a single <code>RiskTeam</code> class with rule-based checks rather than three LLM-powered agents

B.3 Original to Our Team

Component	File(s)	Description
LangGraph StateGraph pipeline	<code>orchestrator.py</code>	7-stage pipeline with 3 conditional edges and 4 termination points
GlobalState typed protocol	<code>models.py</code>	Typed dataclass with append-only traces
Rule-based fallbacks	<code>agents.py</code>	Independent fallback for every LLM-powered agent
Streamlit dashboard	<code>dashboard.py</code>	5-tab interactive interface
CLASSic evaluation framework	<code>evaluation/classic_evaluator.py</code>	All 6 CLASSic elements
Historical backtest engine	<code>backtest.py</code>	3 pinned stress periods, rule-based strategy
Experimental evaluation design	<code>evaluation/eval_runner.py</code>	10 scenarios with failure/adversarial cases
LLM cost / latency tracking	<code>llm_interface.py</code>	Per-call timing, token, cost logging
SQLite persistence	<code>persistence.py</code>	Multi-day portfolio state
Evidence package generator	<code>evaluation/evidence_package.py</code>	Automated evidence collection
Live data integration	<code>tools.py</code>	yfinance + Finnhub with mock fallback
Concentration cap enforcement	<code>agents.py</code> (RiskTeam)	Dual-level enforcement (Trader soft, Risk Team hard)
Human-in-the-loop gate	<code>orchestrator.py</code> , <code>dashboard.py</code>	Interactive in Streamlit, auto-approve in CLI

Summary. Zero components are copied directly. Three high-level concepts are adapted. Thirteen-plus components, including all Phase 3 additions, are original.

C Phase 3 Contribution Update

This appendix documents *forward-looking* Phase 3 ownership in response to the feedback that the Phase 2 update was “backward-looking only.”

C.1 Raj Bhuyan

Area	Deliverable	Status
Backtest infrastructure	<code>backtest.py</code> : 3 historical stress periods, rule-based engine	Complete
CLASSic evaluation	<code>evaluation/classic_evaluator.py</code> : all 6 elements	Complete
Streamlit dashboard	<code>dashboard.py</code> : 5 tabs, interactive human gate	Complete
Live data integration	<code>tools.py</code> : yfinance + Finnhub with mock fallback	Complete
Final report	<code>docs/final_report.md</code> : 8 required sections	Complete

C.2 Sanath Mahesh Kumar

Area	Deliverable	Status
Failure case design	3+ failure scenarios in <code>eval_runner.py</code>	Complete
Evidence package	<code>evaluation/evidence_package.py</code> : automated collection	Complete
Governance reflection	Sections 6 and 7 of the final report	Complete
LLM cost tracking	<code>llm_interface.py</code> : per-call metrics	Complete
Documentation review	AI disclosure, inherited boundary docs	Complete

C.3 Mahir Nagersheth

Area	Deliverable	Status
5-minute video	<code>docs/video_script.md</code> + recording	Script complete
Experimental evaluation	<code>evaluation/eval_runner.py</code> : 10 scenarios	Complete
SQLite persistence	<code>persistence.py</code> : multi-day state	Complete
README and repo cleanup	<code>README.md</code> : updated for Phase 3	Complete
Presentation materials	Slides + demo script	Complete

C.4 Cumulative Hours (Phase 1–3)

Member	Phase 1	Phase 2	Phase 3	Total
Raj Bhuyan	8h	12h	20h	40h
Sanath Mahesh Kumar	8h	12h	18h	38h
Mahir Nagersheth	8h	10h	18h	36h

D AI Usage Disclosure

D.1 Groq API (llama-3.3-70b-versatile): Core System Component

Purpose. Primary LLM powering all agent reasoning within the portfolio management system.

Usage. Powers the Fundamental, Technical, Sentiment, and News Analysts; the Bullish and Bearish Researchers and the Debate Facilitator; the Trader Agent; and the LLM Judge in evaluation.

Mechanism. Each agent sends a structured system prompt and user prompt to the Groq API and parses the response as JSON. If the call fails or the response is malformed, the agent falls back to deterministic rule-based logic.

Transparency. All LLM calls are logged with timing, token usage, and cost estimates. The full call log lives in `sample_runs/llm_metrics.json` and feeds the CLASSic Report.

D.2 GitHub Copilot and AI Coding Assistants

Purpose. Code generation assistance during development.

Usage. Boilerplate (dataclass definitions, Streamlit layout), debugging assistance for Lang-Graph conditional edges, and documentation drafting.

Review. All AI-generated code was reviewed, tested, and modified by a team member. No code was used verbatim without understanding and verification.

D.3 ChatGPT and Claude: Research and Planning

Purpose. Research assistance for architecture design and evaluation methodology.

Usage. Research on LangGraph patterns, the CLASSic evaluation framework, evaluation scenario phrasing, and review of financial methodology (RSI, MACD, Sharpe ratio).

Adaptation. Research outputs informed design decisions but were not copied directly.

D.4 Boundary: AI-Assisted vs. Human-Written

Component	AI assistance level	Human contribution
Architecture design	Low (brainstorming only)	Full design, role definitions, conditional edges
Agent implementations	Medium (boilerplate help)	Logic design, fallback rules, prompt engineering
Evaluation framework	Low (CLASSic research)	Scenario design, threshold selection, analysis
Streamlit dashboard	Medium (layout help)	Tab design, integration logic, UX decisions
Backtest engine	Low	Full implementation of rule-based strategy
Documentation	Medium (drafting aid)	Content decisions, accuracy verification
Risk controls	None	Full design of caps, fallbacks

Final report prepared for the CMU Agentic Systems Studio, Spring 2026.